SOFTWARE PROCESS MANAGEMENT

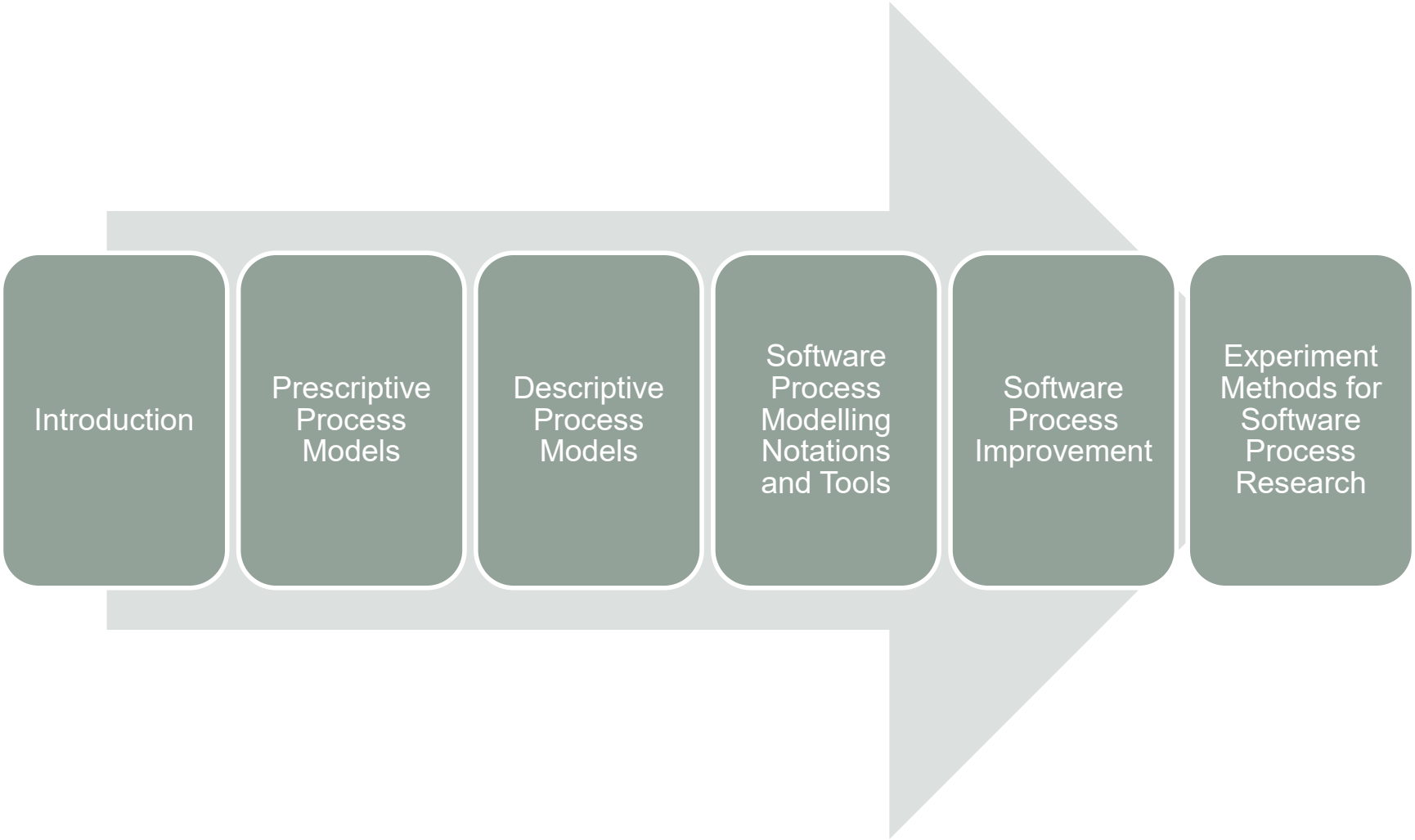
Introduction

Course Aims

After completing this module students will be able to

- Define what a software process is
- Identify main types of software processes
- Define key attributes of a software process
- Model and design a software process
- Evaluate the outcomes of a software process
- Improve a software process

Course Outline



Course Overview

- This module:
- Follows the faculty guideline of OBE (Outcome Based Education)
- Extensively uses student centred learning:
  - Collaborative group learning
  - Self-study through exploration
  - Experiential learning through application

Course Overview

- CA – 40%
  - Group Project
    - Presentation + report
  - Individual Assignment(s)
- Final Examination – 60%
  - 2 hours
  - Closed-book

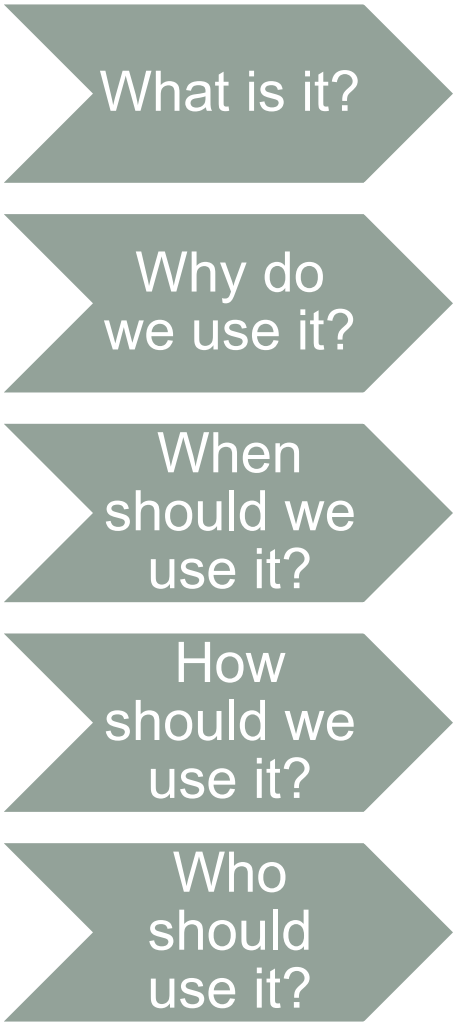
Related Study Areas

- Process Management
- Project Management
- Business Processes
- People Management
- Software Engineering (Life cycle activities)
  - Requirement Engineering
  - Software Design
  - Software Development
  - Software Testing
  - Software Implementation and Maintenance

Software Process Management

- Why study this
- Gives you a competitive advantage along with your technical expertise
- Helps to identify potential challenges and points of failures well in advance, thereby increasing the chances of project success
- You can contribute more for improving software lifecycle activities and quality of life of software engineers / developers
- You may become a central player addressing most of the problems of the stakeholders of the software project

Software Process and Management



A Process

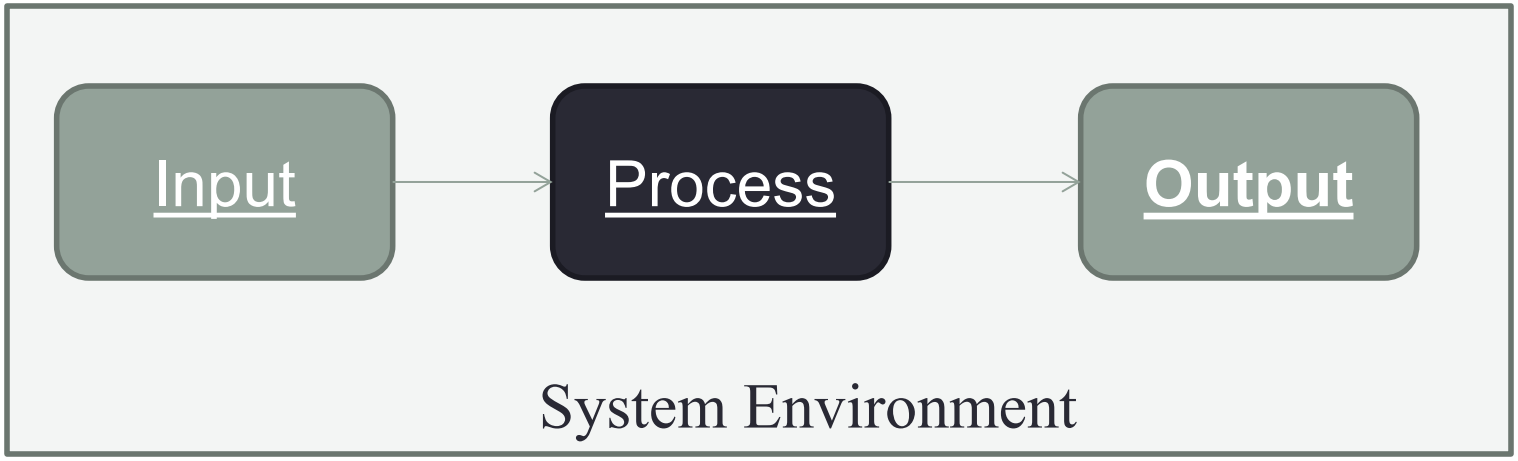
- System View



- What do we have with respect to software?

A Process

- How do we transform input into output?
- How do we define the level of transformation?



A Process

- Involves one or more tasks
- Tasks can be subdivided into smaller tasks
  - Until you get an atomic (indivisible further) actions
- We usually call these tasks, *activities*
- Hence, a process is
  - A collection of activities
  - Right?

Process – definition

- Activities should follow a proper structure
  - If not it becomes chaotic
- Therefore, a process is a structured set of activities
- Hence, a software process is
- *A structured set of activities required to develop a software system*

Software Process Activities

- There are many different software processes but all involve:
  - Specification – defining what the system should do
  - Design and implementation – defining the organization of the system and implementing the system
  - Validation – checking that it does what the customer wants
  - Evolution – changing the system in response to changing customer needs.

Managing The Software Process

- Why should we manage the software process?

- Let’s try to answer this

Important Quotes

- "If you don't know where you are going, any road will do." Chinese Proverb
- "If you don’t know where you are, a map won't help." Watts Humphrey
- "If you don't know where you are going, a map won't get you there any faster." Anonymous ☺
- "You can't expect to be a functional employee in a dysfunctional environment" Watts Humphrey
- If you fail to plan, you are planning to fail Anonymous

Common Misconceptions about the Software Process

- We must start with firm requirements
  - Reality: Use an incremental software process
- If the software passes test, it must be OK
  - Reality: Testing should strive to find errors, not prove they don't exist
- Software quality cannot be measured
  - Reality: There exist many analysis and design metrics
- The problems are technical
  - Reality: Immature organizations continue to make and remake the same management mistakes
- We need better people
  - Reality: Most problems can only be fixed through management action
- Software management is different
  - Reality: Yes at the micro level, but no at the macro level

Why should we manage the software process?

Individuals, Teams, and Armies

- History of software is one of increasing scale
  - Initially a few people could craft small programs
  - Today large projects require the coordinated work of many teams
- The increase in scale requires a more structured approach to software process management

Today, if we talk about Software

- I mean… in real business…



People and the Software Process

- Talented people are the most important element in a software organization
- Successful organizations provide a structured and disciplined environment to do cooperative work
- Alternative
  - Endless hours of repetitively solving technically trivial problems
  - Time is consumed by mountains of uncontrolled detail
- If the details are not managed, the best people cannot be productive
- First class people need the support of an orderly process to do first-class work

A Perspective on the People

- Better people clearly do better work

- However, focusing only on talent can lead into a blind alley
  - The best people are always in short supply
  - You probably have the best team you can get right now
  - With proper leadership and support, most people can do much better work than they are currently doing now

Myth of the Super Programmers

- Common view: First-class people intuitively know how to do first-class work
  - Implication: No orderly process framework is needed
  - Conclusion: Organizations with the best people should not suffer from software quality and productivity problems

- However, studies show that companies with top graduates from leading universities are still plagued with the same problems
  - New Conclusion: The best people need to be supported with an effectively managed software process

No more demand for cowboy coders



Myth of Tools and Technology

- Common View: Some technically advanced tool or method will provide a magic answer to the software crisis

- Reality: Technology is vital, but unthinking reliance on an undefined "silver bullet" will divert attention from the need for better process management

Major Concerns of Software Professionals

- Open-ended requirements
- Uncontrolled change
- Arbitrary schedules
- Insufficient test time
- Inadequate training
- Unmanaged system standards

- Very few even mention technology as a key problem

Limiting Factors in using Software Technology

- Poorly-defined process
- Inconsistent implementation
- Poor process management

Focusing on Software Process Management

- Managed Software process: the set of actions required to efficiently transform a user's need into an effective software solution

- Many software organizations have trouble defining and controlling this process
  - Even though this is where they have the greatest potential for improvement

What makes Software Organizations Chaotic?

- Unplanned commitments
  - Schedules may show what is to be done but not an achievable plan to do the work
- Reliance on gurus
  - Believe they can do no wrong
  - When they fail, there is almost no way for the company to recover
- Belief in magic
  - Humans are repelled by complexity so they try to make details seem so unnecessary that the hard work is deferred while Rome burns
- Problems of scale
  - Having learned to build small programs, we falsely believe we are prepared to build large programs using the same skills

More on Problems of Scale

- As software products become larger, they are much more difficult to understand
- As software knowledge is more widely distributed, common notations are needed, the notations must be documented, conflicts in standards must be resolved, and standards must be controlled and distributed
- With larger-scale software, similar control is needed for requirements analysis, design, coding, and testing
- As software size increases, prototypes or multiple releases are needed
- With multiple releases, more complications arise concerning release schedules and other interdependencies

Managing a Process

- In order to manage you must
  - Control the process activities
- For that one must
  - Monitor the process in progress
- In overall, you must know
  - Why bother → Why bother controlling → Why bother managing
  - Where to monitor → Where to control → Where to manage
  - When to monitor → When to control → When to manage
  - What to monitor → What to control → What to manage
  - How to monitor→ How to control → How to manage

Process Model

- A process itself has input and output (micro level)
- A software process is an intangible (imaginary) concept in which we follow through tangible artefacts and tasks
- Humans do all sorts of funny things when
  - Understanding,
  - Expressing,
  - Executing,
  - Engaging in
- Imaginary tasks and concepts
- Why
  - Each individual is unique with their perception, cognitive skills and attitudes

Process Models

- We are better off with observable concepts, metaphors, and artefacts
- It helps us to grasp cognitively demanding stuff
- E.g.
  - Design plan of a building (house)
  - Different perspectives of a 3D model
  - Software architectural diagrams
    - Structural behaviour
    - Dynamic behaviour

- So, what about software processes?

Software Process Model

- A software process model is an abstract representation of a process.
- It presents a description of a process from some particular perspective.

- Usually a software process is expressed, explained and analysed by using the model that represents the process.

- Usually includes, but not limited to, phases, activities, milestones and deliverable

Software Process Models

- For managing a software process
- We have to consider its process model

- Process model helps to simulate the dynamic and static behaviours of the process
- Can be used to analyse through statistical and formal methods
- Can be examined for process changes and improvements

- A process model incorporates process descriptions

Software Process Descriptions

- When we describe and discuss processes, we usually talk about the activities in these processes such as specifying a data model, designing a user interface, etc. and the ordering of these activities.
- Process descriptions may also include:
  - Products, which are the outcomes of a process activity;
  - Roles, which reflect the responsibilities of the people involved in the process;
  - Pre- and post-conditions, which are statements that are true before and after a process activity has been enacted or a product produced.

In overall



If you master it well



OR



- Can be a common gesture for the rest of your life ☹

